Phase 1: High-Performance Data Engineering & Analytics

PHASE 1 NOTES | PRODUCTION READY | NYC & E-COMMERCE

Crystal-Clear Revision Handbook • Ingestion, Analytical SQL, Vectorized Wrangling & Data Quality

STEP 01 Memory & Ingestion (Downcasting, Parquet, Chunking)

STEP 02 Analytical SQL (Case When, Windows, CTEs, Cohorts)

STEP 03 Vectorized Wrangling (MultiIndex, Regex, Transforms, Reshape)

STEP 04 Data Quality & Cleansing (MCAR/MAR/MNAR, KNN, Winsor, Fuzzy)

STEP 1: High-Performance Ingestion & Memory Optimization

Scaling Pandas to process millions of records without Out-Of-Memory (OOM) operating system crashes

STEP 1 • PART 1/2

Topic 1.1 Data Type Downcasting & The 5% Cardinality Rule

60% – 85% RAM SAVED

The Problem: By default, Pandas reads numeric columns as 64-bit (`int64` / `float64`), using 8 bytes per cell regardless of range. Text strings are stored as Python `object` pointers with huge memory overhead.

The Solution: 1. *Downcasting:* Dynamically downcast to `int8` (-128 to 127), `int16`, `int32`, or `float32` based on actual min/max values.
2. *The 5% Category Rule:* If `(nunique() / len(df)) < 0.05`, convert string objects to `category`. Stores 1-byte integer IDs pointing to a single lookup table.

Key Takeaway: Run downcasting immediately upon reading raw CSVs before executing any transformations to prevent notebook memory crashes.

PYTHON (PANDAS) — DOWNCASTING IMPLEMENTATION

```python
# 1. Downcast Integers & Floats automatically
df['price'] = pd.to_numeric(df['price'], downcast='integer')
df['latitude'] = pd.to_numeric(df['latitude'], downcast='float')

# 2. Check Cardinality Ratio & Convert Repeated Text
str_cols = df.select_dtypes(include=['object']).columns
for col in str_cols:
    unique_ratio = (df[col].nunique() / len(df)) * 100
    if unique_ratio < 5.0:
        df[col] = df[col].astype('category') # Slashes pointer overhead
```

- `downcast='integer'`: Checks minimum and maximum values and selects the smallest safe C integer.
- `astype('category')`: Replaces 8-byte pointer per row with a 1-byte integer code + dictionary lookup.

Topic 1.2 Columnar Storage: Apache Parquet vs. CSV

10X – 50X FASTER • 70% DISK SAVED

The Problem: CSV is row-based, uncompressed plain text. To read just 1 column (e.g. `price`), Pandas must scan and parse every byte of all columns across the entire hard drive.

The Solution (Columnar + Compression): Parquet stores data column-by-column, compressed with Snappy, and embeds metadata (byte offsets, min/max stats) in a footer.

Column Pruning: Parquet reads *only* the requested byte ranges directly from disk, skipping all unwanted columns completely.

Key Takeaway: In enterprise pipelines, convert landing CSVs to Parquet immediately. Read only necessary columns using `columns=['col']`.

PYTHON (PYARROW / PARQUET) — STORAGE BENCHMARK

```python
# Save Dataframe as Parquet with Snappy Compression
df.to_parquet('nyc_data.parquet', engine='pyarrow', index=False)

# Column Pruning: Reads ONLY 'price' without touching other columns
df_sub = pd.read_parquet('nyc_data.parquet', columns=['price'])

# Benchmark:
# CSV on Disk:     ~7.08 MB (Raw uncompressed text)
# Parquet on Disk: ~2.44 MB (65.5% Disk Savings + Schema Preserved!)
```

- `engine='pyarrow'`: High-performance C++ columnar reader/writer.
- `columns=['price']`: Performs zero disk I/O on unrequested columns.

Topic 1.3

Chunking: Processing Massive Files Without OOM

CONSTANT RAM CEILING

The Problem: Loading a 20GB dataset onto an 8GB machine results in an immediate Out-Of-Memory (OOM) operating system crash.

The Solution: Process the dataset as a stream in discrete slices using `chunksize`. Each chunk is loaded, aggregated, and immediately evicted from memory.

Memory Guarantee: RAM usage stays flat (e.g. 20MB) regardless of whether the underlying CSV is 100MB or 500 Gigabytes!

Golden Rule: Never append raw chunks into a list in RAM; maintain lightweight running scalars (sum, count, min, max) inside the loop.

PYTHON (PANDAS) — STREAMING CHUNK ITERATOR

```python
chunk_size = 50_000
total_revenue = 0.0
total_orders = 0

# Returns an iterator yielding DataFrames of chunk_size rows
for chunk in pd.read_csv('huge_orders.csv', chunksize=chunk_size):
    total_revenue += chunk['price'].sum()   # Accumulate scalar state
    total_orders += len(chunk)              # Prior chunk freed by GC

print(f"Orders: {total_orders:,} | Revenue: ${total_revenue:,.2f}")
```

> `chunksize=N`: Converts `read_csv` into a Python generator.
> Enables constant RAM ceiling and prevents OOM fatal errors.

Step 1 Synthesis

Ingestion Formats & Memory Decision Framework

ARCHITECTURE GUIDE

Storage / Mode	Disk Footprint	RAM Usage	Query Speed	When to Use (Engineering Rule)
Raw CSV	100% (Baseline)	Heavy (64-bit default)	Slow (Full row parse)	Only for initial data ingestion or human-readable inspection.
Downcasted DF	N/A (In-Memory)	**20% – 35% of original**	Fast (Fits in CPU cache)	Mandatory first step when working with Pandas in RAM.
Apache Parquet	**25% – 35% of CSV**	Minimal (Column Pruning)	**10x – 50x faster**	Default enterprise analytical storage; best for cloud datalakes.
Chunked Stream	100% on disk	**Constant ceiling (20MB)**	Linear streaming	Mandatory when dataset file size exceeds available machine RAM.
SQLite In-Memory	N/A (RAM-backed)	Lightweight binary	Index-accelerated	Performing complex SQL joins and window functions without a server.

Decision Rule 1 (Data Volume):
• File < 50% RAM: Load directly → Downcast types → Convert low cardinality to `category`.
• File > 50% RAM: Use `chunksize` or convert to Parquet and load specific columns.

Decision Rule 2 (Query Pattern):
• Full row ETL: Parquet or Arrow streaming.
• Ad-hoc aggregations: Parquet columnar reading with `columns=['col']` pruning.

Topic 2.1 Conditional Aggregations (Single-Pass CASE WHEN)

1 SCAN VS 5 SCANS

The Problem: Running multiple separate queries with different `WHERE` clauses or executing complex `SELF JOIN`s to aggregate different segments causes multiple redundant table scans.

The Solution: Embed `CASE WHEN` inside aggregation functions (`SUM`, `AVG`). This evaluates all segments simultaneously in a *single sequential scan*.

> **Crucial SQL Syntax Rule:** In `AVG(CASE WHEN col='A' THEN val END)`, omit the `ELSE`. SQL treats omitted conditions as `NULL`, and `AVG()` ignores `NULL`s, computing the true segment average!

> **Impact:** Reduces query execution time from $O(N imes K)$ table scans down to a single $O(N)$ pass.

SQL (SQLITE / POSTGRES / BIGQUERY) — SINGLE-PASS AGGREGATION

```sql
SELECT
    neighbourhood_group,
    COUNT(*) AS total_listings,
    -- Conditional Counting
    SUM(CASE WHEN room_type = 'Entire home/apt' THEN 1 ELSE 0 END) AS ent
ire_homes,
    SUM(CASE WHEN room_type = 'Private room' THEN 1 ELSE 0 END) AS privat
e_rooms,
    -- Segment Averages (No ELSE -> NULL ignored by AVG)
    ROUND(AVG(CASE WHEN room_type = 'Entire home/apt' THEN price END), 2)
AS avg_entire_price,
    ROUND(AVG(CASE WHEN room_type = 'Private room' THEN price END), 2) AS
avg_private_price
FROM airbnb
GROUP BY neighbourhood_group;
```

Topic 2.2 Window Functions (OVER PARTITION BY)

ZERO ROW COLLAPSE

The Core Distinction: `GROUP BY` collapses multiple rows into 1 summary row. Window functions (`OVER (PARTITION BY ... ORDER BY ...)`) calculate group stats and rankings *without collapsing*, preserving every individual row's granularity.

Key Window Operations:

- `ROW_NUMBER()`: Sequential unique number (1, 2, 3...).
- `DENSE_RANK()`: Rank with ties, never skips numbers (1, 2, 2, 3).
- `LAG(col, N) / LEAD(col, N)`: Fetch prior/next row's value.
- `SUM(col) OVER(...)`: Cumulative running total.

> **Key Takeaway:** To filter on window ranks, wrap the query in a CTE and filter in the outer `WHERE` clause (since window functions execute after `WHERE`).

SQL — TOP-N & GROUP BENCHMARKING

```sql
WITH ranked_listings AS (
    SELECT
        id, neighbourhood_group, price,
        -- 1. Rank prices inside each borough
        DENSE_RANK() OVER (
            PARTITION BY neighbourhood_group
            ORDER BY price DESC
        ) AS price_rank,
        -- 2. Attach borough average to every individual listing
        ROUND(AVG(price) OVER (PARTITION BY neighbourhood_group), 2) AS b
orough_avg
    FROM airbnb
)
SELECT * FROM ranked_listings
WHERE price_rank <= 2; -- Top 2 highest-priced listings per borough
```

Topic 2.3 Recursive CTEs: Traversing Hierarchical Trees

GRAPH / TREE TRAVERSAL

The Problem: Relational tables store hierarchical tree relationships (e.g., Employee -> Manager, Category -> Subcategory) where depth is dynamic and unknown. Standard SQL joins cannot traverse unknown depths.

The Architecture of a Recursive CTE: 1. *Anchor Member:* The starting query (e.g. CEO who has `manager_id IS NULL`).
2. `UNION ALL`: Combines the anchor with the loop.
3. *Recursive Member:* Joins child rows to the previous parent result until no more rows match.

SQL (RECURSIVE CTE) — ORG HIERARCHY & REPORTING PATH

```sql
WITH RECURSIVE org_tree AS (
    -- 1. ANCHOR: Top root of the hierarchy (CEO)
    SELECT emp_id, emp_name, manager_id, 1 AS depth, emp_name AS path
    FROM employees WHERE manager_id IS NULL

    UNION ALL

    -- 2. RECURSIVE STEP: Find subordinates reporting to prior depth
    SELECT e.emp_id, e.emp_name, e.manager_id, t.depth + 1,
           t.path || ' -> ' || e.emp_name
    FROM employees e
    JOIN org_tree t ON e.manager_id = t.emp_id
)
SELECT emp_id, emp_name, depth, path FROM org_tree;
```

Topic 2.4 Cohort & Longitudinal Matrix (Retention Analysis)

ENTERPRISE ANALYTICS PATTERN

What is a Cohort? A group of entities sharing a common starting event (e.g., signup month, first order). Tracking them over time measures retention and product stickiness.

The 3-Step SQL Retention Pattern: 1. *Identify Cohort Birth:* Find `MIN(date)` per user.
2. *Compute Period Offset:* `(Activity Period - Cohort Birth Period)`.
3. *Pivot to Grid:* Use `CASE WHEN offset = 0, 1, 2...` to create the horizontal retention triangle.

SQL — COHORT RETENTION GRID

```sql
WITH first_activity AS (
    SELECT user_id, MIN(login_month) AS cohort_month
    FROM user_logins GROUP BY user_id
),
cohort_deltas AS (
    SELECT f.cohort_month,
           (CAST(SUBSTR(l.login_month, 6, 2) AS INT) -
            CAST(SUBSTR(f.cohort_month, 6, 2) AS INT)) AS month_offset,
           COUNT(DISTINCT l.user_id) AS active_users
    FROM user_logins l JOIN first_activity f ON l.user_id = f.user_id
    GROUP BY f.cohort_month, month_offset
)
SELECT cohort_month,
    SUM(CASE WHEN month_offset = 0 THEN active_users ELSE 0 END) AS m_0,
    SUM(CASE WHEN month_offset = 1 THEN active_users ELSE 0 END) AS m_1
FROM cohort_deltas GROUP BY cohort_month;
```

SQL Secret True Logical Execution Order of a Query

INTERVIEW HIGH-YIELD

Execution Hierarchy: `1. FROM & JOIN → 2. WHERE → 3. GROUP BY → 4. HAVING → 5. WINDOW (OVER) → 6. SELECT → 7. DISTINCT → 8. ORDER BY → 9. LIMIT`.

Why this explains the "Window Filter" error: Because `WHERE` executes at Step 2 and `WINDOW` executes at Step 5, you **cannot** filter window results directly in a `WHERE` clause. You must wrap the window in a CTE first!

Topic 3.1 MultiIndex Slicing & Cross-Sections (.xs)

HIGH-DIMENSIONAL FAST LOOKUP

The Problem: Slicing datasets on multiple keys with boolean conditions (`(df['borough']=='Manhattan') & (df['type']=='Entire home')`) performs slow full-array comparisons every time.

The Solution: Create a hierarchical `MultiIndex`. 1. *Direct Tuple Lookup:* `.loc[('Manhattan', 'Entire home/apt')]` uses indexed binary search.
2. *Cross-Section (.xs):* Extract all records of an inner level across ALL outer levels without knowing the outer keys!

> **Golden Rule:** Always call `.sort_index()` after setting a MultiIndex. Unsorted MultiIndexes trigger performance warnings and slow lookups.

PYTHON (PANDAS) — MULTIINDEX & CROSS SECTION

```python
# Create sorted two-level index: Borough -> Room Type
df = df.set_index(['neighbourhood_group', 'room_type']).sort_index()

# 1. Exact Tuple Lookup (Fast Indexed Slice)
manhattan_entire = df.loc[('Manhattan', 'Entire home/apt')]

# 2. Cross-Section (.xs): Pull 'Private room' across ALL boroughs!
all_private_rooms = df.xs('Private room', level='room_type')

# Reset back to flat table when done
df = df.reset_index()
```

Topic 3.2 Vectorized String & Named Regex Extraction (.str.extract)

COMPILED C-SPEED TEXT PARSING

The Problem: Using Python `for` loops or `df['col'].apply(lambda x: re.search(...))` runs in slow Python interpreter space, taking minutes on large datasets.

The Solution: `.str.extract()` compiles the regular expression in C. Named capture groups `(?P<column_name>pattern)` directly become the column headers of a brand new DataFrame.

> **Non-Capturing Groups:** Use `(?: ... )?` for optional qualifiers so they don't produce extra unwanted columns.

PYTHON (REGEX VECTORIZATION) — EXTRACT NAMED COLUMNS

```python
# Parse bedrooms and property types from raw listing names
pattern = (
    r'(?i)(?:(?P<bedroom>\d)\s*(?:br|bedroom|bed))?.*?'
    r'\b(?P<property_tag>Studio|Loft|Brownstone|Apartment|Apt|Suite|Room)\b'
)

# Extracts 2 clean columns simultaneously at compiled C speed
extracted_df = df['name'].str.extract(pattern)

# Retain rows where property_tag is found (even if bedroom count is NaN)
valid_tags = extracted_df.dropna(subset=['property_tag'])
```

Topic 3.3 Custom Mapping Pipelines: .transform() vs .agg()

BROADCASTING ROW BENCHMARKS

The Critical Distinction:

Method	Behavior	Output Size
`.agg()`	Collapses rows into group summaries	N groups
`.transform()`	Computes group stat and broadcasts it back	Matches original N rows

Calculating Group Z-Scores: Benchmark each row against its own group average and standard deviation without performing slow merges or joins:
`Z = (price - borough_avg) / borough_std`

PYTHON (PANDAS) — TRANSFORM BROADCASTING PIPELINE

```python
# 1. Broadcast group mean and standard deviation back to every row
df['borough_avg'] = df.groupby('neighbourhood_group')['price'].transform('mean').round(2)
df['borough_std'] = df.groupby('neighbourhood_group')['price'].transform('std').round(2)

# 2. Compute Row-Level Deviation and Group Z-Score
df['diff_from_avg'] = (df['price'] - df['borough_avg']).round(2)
df['price_zscore'] = (df['diff_from_avg'] / df['borough_std']).round(2)
# Rows with zscore > +3.0 are extreme borough outliers!
```

Topic 3.4 Reshaping: Pivot Tables, Melt (Tidy Long), & Rolling Windows

DIMENSIONAL FLEXIBILITY

Wide vs. Tidy (Long) Format: 1. *Wide Matrix (Pivot):* Converts categories into columns. Best for executive reports & heatmaps.
2. *Long Tidy Format (Melt):* Every row is 1 observation, columns are variables. **Mandatory** for Seaborn, Plotly, and ML feature stores.
3. *Rolling Windows:* Smooths erratic volatile metrics over time sequences.

PYTHON (PANDAS) — PIVOT, MELT & ROLLING CALCULATION

```python
# 1. Pivot Table (Convert Long records to Wide Summary Matrix)
wide_matrix = df.pivot_table(
    index='neighbourhood_group', columns='room_type', values='price', aggfunc='mean'
).round(2)

# 2. Melt (Unpivot Wide Matrix back into Tidy Long format)
tidy_long = pd.melt(wide_matrix.reset_index(), id_vars='neighbourhood_group',
                    var_name='room_type', value_name='avg_price')

# 3. Rolling 7-Review Moving Average Window
df_dated = df.dropna(subset=['last_review']).sort_values('last_review')
df_dated['rolling_7_price'] = df_dated['price'].rolling(window=7, min_periods=1).mean().round(2)
```

Step 3 Cheat Sheet The Dimensional Reshaping Cheat Sheet

PIPELINE STANDARD

Transformation	Method Call	Input → Output	Ideal Target Use Case
Pivot / Cross-tab	`df.pivot_table(index, cols, vals)`	Long (Tidy) → Wide Grid	Executive KPI matrices, heatmaps, cohort grids.
Melt (Unpivot)	`pd.melt(df, id_vars, var_name)`	Wide Grid → Long (Tidy)	Plotting libraries (Seaborn, Plotly) & ML feature stores.
Stack / Unstack	`df.stack() / df.unstack()`	MultiIndex ↔ Wide Columns	Pivoting dimensional hierarchies without aggregation.

Topic 4.1 The Missing Data Taxonomy: MCAR vs. MAR vs. MNAR

STATISTICAL DIAGNOSIS

Type	Mechanism & Definition	Real-World Example	Correct Engineering Action
MCAR Completely at Random	Probability of missingness is purely random and independent of observed and unobserved data.	Sensor communication glitch, dropped packet, coffee spilled on paper form.	Safe to drop rows or impute with simple median/mean. No statistical bias added.
MAR Missing at Random	Missingness is systematically explained by *other observed variables* in the dataset.	`reviews_per_month` is NaN only when `number_of_reviews == 0`; students have blank income.	Conditional domain filling (`fillna(0.0)`) or multivariate KNN Imputation.
MNAR Not at Random	Missingness depends on the *unobserved value itself*. Highly biased.	Extremely dissatisfied customers skip satisfaction surveys; high-earners hide wealth.	**NEVER impute blindly!** Always create an indicator flag: `df['col_was_missing'] = isnull()`.

PYTHON (PANDAS) — DIAGNOSING & RESOLVING MAR & MNAR

```python
# 1. Diagnose MAR: Check correlation between missingness and observed features
is_mar = df['reviews_per_month'].isnull().corr(df['number_of_reviews'] == 0)
# Strong correlation (+1.0) proves MAR: If 0 reviews exist, monthly rate is logically 0.0
df['reviews_per_month'] = df['reviews_per_month'].fillna(0.0)

# 2. Handle MNAR: Always preserve the missingness signal with an indicator column!
df['satisfaction_was_missing'] = df['satisfaction_rating'].isnull().astype(int)
```

Topic 4.2 Advanced Imputation: KNN Imputer vs. Simple Mean

MULTIVARIATE CONTEXT AWARENESS

Why Simple Mean Fails: Imputing missing values with global mean collapses the feature variance, creates artificial spikes in distributions, and completely ignores correlations with other features.

The KNN Imputer Advantage: Finds the k most similar records across multi-dimensional feature space (e.g., latitude, longitude, nights) using Euclidean distance and calculates their weighted mean.

> **Rule:** In geospatial and pricing datasets, nearby properties have similar prices. KNN preserves spatial clustering!

PYTHON (SCIKIT-LEARN) — KNN IMPUTATION

```python
from sklearn.impute import KNNImputer

# Select numerical features for distance calculation
features = ['price', 'latitude', 'longitude', 'minimum_nights']
knn = KNNImputer(n_neighbors=3, weights='distance')

# Fits Euclidean distance matrix & replaces NaNs accurately
imputed_array = knn.fit_transform(df[features])
df['price_imputed'] = np.round(imputed_array[:, 0], 2)

# Result: Imputed price reflects real neighborhood value, not citywide mean!
```

Topic 4.3 Outlier Control: Winsorization (Percentile Clipping)

ZERO DATA LOSS • VARIANCE STABILIZED

The Dilemma: Deleting extreme outliers (e.g. $10,000/night penthouse) causes sample loss and survivorship bias. Leaving them uncontrolled skews linear regressions, neural networks, and standard deviations.

Winsorization Solution: Cap values at 1st and 99th percentiles. Extreme values are clamped to the boundary without dropping a single observation.

> **Measured Result:** NYC listing prices dropped standard deviation by over 50% ($240 down to $118), instantly stabilizing downstream ML models!

PYTHON (PANDAS) — PERCENTILE CLIPPING / WINSORIZATION

```python
# 1. Calculate 1st and 99th percentile statistical cutoff thresholds
lower_bound = df['price'].quantile(0.01)  # e.g., $30.00
upper_bound = df['price'].quantile(0.99)  # e.g., $799.00

# 2. Clip extremes while retaining 100% of rows
df['price_winsorized'] = df['price'].clip(
    lower=lower_bound,
    upper=upper_bound
)

# Minimum becomes $30, Maximum becomes $799; all extreme spikes tamed!
```

Topic 4.4 Malformed Data Resolution: Fuzzy String Matching

LEVENSHTEIN EDIT DISTANCE

The Problem: User inputs contain typos, misspellings, and inconsistent casing ("manhatten", "Brookln", "the bronx"). Exact SQL/Pandas joins fail completely.

The Solution: Calculate Levenshtein distance (minimum character insertions, deletions, or substitutions) against a canonical gold-standard dictionary using Python's built-in `difflib`.

> **Production Tip:** Set `cutoff=0.6`. If score < 0.6, flag as "Unmatched" for human QA rather than making false automated matches.

PYTHON (STANDARD LIBRARY DIFFLIB) — FUZZY MATCHING

```python
import difflib

# Official canonical categories in database
canonical_boroughs = ['Manhattan', 'Brooklyn', 'Queens', 'Bronx', 'Staten Island']
messy_inputs = ['manhatten', 'Brookln', 'Quens', 'the bronx', 'Unknown Place']

for dirty in messy_inputs:
    # Find best match with at least 50% similarity ratio
    matches = difflib.get_close_matches(dirty, canonical_boroughs, n=1, cutoff=0.5)
    cleaned = matches[0] if matches else "Unmatched / Review"
    # Result: 'manhatten' -> 'Manhattan' | 'Brookln' -> 'Brooklyn'
```

Master Summary Phase 1 Golden Rules & Interview Cheatsheet Matrix

EXECUTIVE QUICK-REVIEW

Operation	Code / Syntax	Performance Impact / Best Practice
Downcast Int/Float	`pd.to_numeric(col, downcast='integer'/'float')`	Slashes RAM footprint by 50%–75%; preserves original precision.
Categorical Conversion	`col.astype('category')` if unique ratio < 5%	Replaces memory-heavy string pointers with compact 1-byte integer codes.
Columnar Parquet	`pd.read_parquet('file.parquet', columns=['col'])`	10x–50x speedup via Column Pruning; saves ~70% disk space vs CSV.
Streaming Chunking	`for chunk in pd.read_csv(file, chunksize=50000):`	Guarantees constant memory consumption; processes 100GB+ files on 8GB RAM.
Conditional SQL Agg	`SUM/AVG(CASE WHEN cond THEN val END)`	Replaces 3-5 slow queries or joins with 1 single high-speed table scan.
Window Partitioning	`DENSE_RANK() OVER (PARTITION BY grp ORDER BY val)`	Ranks and computes running totals without collapsing individual records.
MultiIndex Lookup	`.set_index(['a','b']).sort_index().xs('val', level='b')`	Binary-search indexed lookups; fast cross-section slicing across hierarchies.
Broadcast Transform	`df.groupby('grp')['col'].transform('mean')`	Attaches group benchmarks directly to original rows without `.merge()`.
Taxonomy: MAR vs MNAR	`df['col'].fillna(0.0)` (MAR) vs `col.isnull()` flag (MNAR)	Preserves statistical validity; prevents distorting ML feature distributions.
KNN Imputation	`KNNImputer(n_neighbors=3).fit_transform(features)`	Uses multi-feature Euclidean distance; avoids variance collapse of mean.
Outlier Winsorizing	`df['col'].clip(lower=q01, upper=q99)`	Preserves 100% of data points; cuts standard deviation by half.
Fuzzy Text Matching	`difflib.get_close_matches(input, canonical, cutoff=0.5)`	Resolves typos and messy user inputs to standardized database categories.